

AOULINK

Smart Contract Security Audit

Audited by



No.202605251611

May 25th, 2026



Contents

1 Overview	5
1.1 Project Overview	5
1.2 Audit Overview	5
1.3 Audit Method	5
2 Findings	7
[AOULINK-01] Centralization Risk	8
[AOULINK-02] Fund Flow Mismatch	9
[AOULINK-03] Absence of Emergency Pause Mechanism	10
[AOULINK-04] Unbounded Rate Parameters	11
3 Appendix	12
3.1 Vulnerability Assessment Metrics and Status in Smart Contracts	12
3.2 Audit Categories	15
3.4 About KEKKAI	18



Code Is Law



Summary of Audit Results

After auditing, 3 Low-risks and 1 Info items were identified in the AOULINK project. Specific audit details will be presented in the Findings section. Users should pay attention to the following aspects when interacting with this project:

Low

Fixed : 0 Acknowledged : 3

Info

Fixed : 0 Acknowledged : 1



● Project Description:

Business overview

The project under audit is a USDC staking yield management system deployed on an EVM-compatible chain, comprising three core smart contracts: StakingVault, StakingConfig, and StakingTypes. The system is built around USDC staking, where users deposit USDC to create staking orders. Yields are settled on a daily basis, generating both static rewards and reinvestment rewards, which can be distributed and withdrawn in the form of USDC or AAU tokens.

StakingVault serves as the core vault contract, managing the full lifecycle of staking orders from creation through final withdrawal. Each vault instance is bound to a single user address, which is either automatically assigned upon the first order creation or pre-configured by the owner, and cannot be changed once set. When creating an order, the caller specifies a tier, a deposit amount, whether auto-renewal is enabled, and whether compounding is active. The system transfers USDC from the user's account into the contract and persists the order details. Settlement follows a bucket-based mechanism, computing rewards at one-day intervals: static rewards are calculated as the principal multiplied by the daily yield rate (`dailyRateBps`), and by default accumulate as withdrawable USDC. If compounding is enabled, static rewards are redirected into the reinvestment balance, which in turn accrues AAU token rewards based on the reinvestment rate (`reinvestRateBps`). If the static-to-AAU mode is active, static rewards are converted directly into AAU tokens at the prevailing AAU price. Order status transitions sequentially through ACTIVE, MATURED, and CLAIMED. Upon maturity, the user may withdraw the full principal along with accrued rewards via on-chain USDC transfer to the user's address, USDC transfer to a designated address, or off-chain BSC USDT payout. To mitigate potential out-of-gas issues caused by prolonged unsettled periods, the system provides a bounded settlement function `processOrderBatch`, which accepts a `maxBuckets` parameter to cap the number of buckets processed in a single transaction.

The StakingConfig contract manages all configurable system parameters, including the daily



Code Is Law



yield rate, reinvestment rate, AAU price, minimum stake amount, tier definitions, and the maximum number of AAU mode switches. To ensure that historical orders are settled using the correct parameters following a rate change, the contract employs a settlement checkpoint mechanism (SettlementCheckpoint). Each parameter update appends a snapshot of the current rates and AAU price to a chronologically ordered array; during settlement, the appropriate checkpoint is resolved via reverse lookup based on the bucket's timestamp. Tier configurations define the lock-up duration and renewal duration for staking orders; when the renewal duration is set to zero, the primary duration is used as the renewal period. All configuration modifications are restricted to the owner role.

The system implements a three-tier access control model. The owner holds the highest privilege level, with the ability to create administrative orders (without transferring USDC), revoke orders and return funds to the user, update global configuration parameters, and sweep arbitrary tokens held by the contract. The user, as the bound account holder of the vault, is authorized to create orders, renew or upgrade order tiers, top up existing orders, toggle reward modes, and withdraw accrued rewards. The reviewer role is dedicated to approving AAU mode switch requests; when a user wishes to toggle between USDC static rewards and AAU rewards, a request must be submitted and approved by the reviewer before taking effect. The system enforces a maximum switch count to prevent abuse.



1 Overview

1.1 Project Overview

Project Name	AOULINK
Project Language	Solidity
File hash	StakingConfig.sol: B00F556BC1D6E0B7C5EBD35D03ACF4F1AF0DB84A6169B5B213C0CF677172CC87
	StakingTypes.sol: CEB892C4CC997D825EF3C3B0C434CDE388AB9244178C4D084D9980E9891EC669
	StakingVault.sol: F4BED3CFECA9F9911889E86A9A45AEF8F48343332221F067FA1BBC6C47A7196A

1.2 Audit Overview

Audit work duration: May 21 – May 25, 2026

Audit team: KEKKAI Security Team

1.3 Audit Method

The audit methods are as follows:

-Formal Verification

Formal verification is a technique that uses property-based approaches for testing and verification. Property specifications define a set of rules using library of security expert rules. These rules call into the contracts under analysis and make various assertions about their behavior. The rules of the specification play a crucial role in the analysis. If the rule is violated, a concrete test case is provided to demonstrate the violation.

-Manual Review

Using manual auditing methods, the code is read line by line to identify potential security issues. This ensures that the contract's execution logic aligns with the client's specifications



Code Is Law



and intentions, thereby safeguarding the accuracy of the contract's business logic.

The manual audit is divided into three groups to cover the entire auditing process:

The Basic Testing Group is primarily responsible for interpreting the project's code and conducting comprehensive functional testing.

The Simulated Attack Group is responsible for analyzing the audited project based on the collected historical audit vulnerability database and security incident attack models. They identify potential attack vectors and collaborate with the Basic Testing Group to conduct simulated attack tests.

The Expert Analysis Group is responsible for analyzing the overall project design, interactions with third parties, and security risks in the on-chain operational environment. They also conduct a review of the entire audit findings.



Code Is Law



2 Findings

Index	Risk description	Severity level	Status
AOULINK-01	Centralization Risk	Low	Acknowledged
AOULINK-02	Fund Flow Mismatch	Low	Acknowledged
AOULINK-03	Absence of Emergency Pause Mechanism	Low	Acknowledged
AOULINK-04	Unbounded Rate Parameters	Info	Acknowledged



Finding Details:

[AOULINK-01] Centralization Risk

Severity Level	Low
Type	Business Security
Lines	StakingVault.sol #L281-289
Description	In the StakingVault contract, the administrator is able to create orders via <code>createOrderByAdmin</code> without transferring any principal. In the event of a private key compromise, an attacker could drain the entire pool by creating fraudulent orders and withdrawing the associated funds. <pre>function createOrderByAdmin(uint256 tierId, uint256 amount, bool autoRenew, bool compoundEnabled) external onlyOwner nonReentrant returns (uint256 orderId) { StakingTypes.TierConfig memory tier = _validateNewOrder(tierId, amount, autoRenew); orderId = _storeOrder(tierId, amount, autoRenew, compoundEnabled, tier, user, msg.sender, true); }</pre>
Recommendation	It is recommended to manage the Owner private key using a multi-signature wallet or equivalent access control mechanism.
Status	Acknowledged.



[AOULINK-02] Fund Flow Mismatch

Severity Level	Low
Type	Business Security
Lines	StakingVault.sol #L263-279
Description	In the StakingVault contract, when a user calls <code>createOrder</code>, the principal is held within the Vault contract itself. However, subsequent withdrawals (e.g., <code>withdrawMaturedOrder</code>) release funds from the ReleasePool rather than the Vault. No on-chain mechanism exists to bridge funds from the Vault to the Pool, which may result in a liquidity shortfall rendering withdrawals impossible. <pre>function createOrder(uint256 tierId, uint256 amount, bool autoRenew, bool compoundEnabled) external nonReentrant returns (uint256 orderId) { if (user != address(0) && msg.sender != user) revert NotUser(); StakingTypes.TierConfig memory tier = _validateNewOrder(tierId, amount, autoRenew); IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), amount); if (user == address(0)) { user = msg.sender; emit UserSet(msg.sender, msg.sender);} orderId = _storeOrder(tierId, amount, autoRenew, compoundEnabled, tier, user, msg.sender, false);} }</pre>
Recommendation	It is recommended to route user deposits directly into the ReleasePool at the time of staking, or to implement an on-chain transfer mechanism between the Vault and the Pool, in order to eliminate the risk of fund flow discontinuity.
Status	Acknowledged.



Code Is Law



[AOULINK-03] Absence of Emergency Pause Mechanism

Severity Level	Low
Type	Business Security
Lines	StakingVault.sol, StakingConfig.sol
Description	Neither the StakingVault nor the StakingConfig contract implements a pause mechanism. In the event of a security incident, there is no way to halt contract operations to mitigate further damage.
Recommendation	It is recommended to integrate OpenZeppelin's Pausable module (or equivalent) to enable emergency suspension of critical functions.
Status	Acknowledged.



[AOULINK-04] Unbounded Rate Parameters

Severity Level	Info
Type	Business Security
Lines	StakingConfig.sol
Description	In the StakingConfig contract, rate-setting functions such as <code>setDailyRateBps</code> do not enforce any upper-bound validation. This allows rate parameters to be set to arbitrarily large values, which could lead to anomalous reward calculations during settlement.
Recommendation	It is recommended to introduce upper-bound checks for all rate parameters to prevent misconfiguration.
Status	Acknowledged.



3 Appendix

3.1 Vulnerability Assessment Metrics and Status in Smart Contracts

3.1.1 Metrics

In order to objectively assess the severity level of vulnerabilities in blockchain systems, this report provides detailed assessment metrics for security vulnerabilities in smart contracts with reference to CVSS 3.1 (Common Vulnerability Scoring System Ver 3.1).

According to the severity level of vulnerability, the vulnerabilities are classified into four levels: "critical", "high", "medium" and "low". It mainly relies on the degree of impact and likelihood of exploitation of the vulnerability, supplemented by other comprehensive factors to determine of the severity level.

Impact Likelihood	Severe	High	Medium	Low
Probable	Critical	High	Medium	Low
Possible	High	Medium	Medium	Low
Unlikely	Medium	Medium	Low	Info
Rare	Low	Low	Info	Info



3.1.2 Degree of impact

- **Severe**

Severe impact vulnerabilities are those that can significantly compromise the confidentiality, integrity, and availability of smart contracts or their economic frameworks. These vulnerabilities can lead to major financial losses within the contract business system, extensive data breaches, loss of administrative control, failure of crucial functionalities, erosion of trust, or even indirectly impact other connected smart contracts, leading to extensive damages. Such vulnerabilities often result in severe and, in many cases, irreversible harm.

- **High**

Vulnerabilities with a high impact are those that can cause considerable damage to the confidentiality, integrity, and availability of smart contracts or their economic models. These can lead to substantial economic losses, localized dysfunctionality, erosion of trust, and other significant detrimental effects on the contract business system.

- **Medium**

Medium impact vulnerabilities are those that can moderately affect the confidentiality, integrity, and availability of smart contracts or their economic models. These vulnerabilities might lead to minor financial losses, affect individual business operations, and have other moderate repercussions on the contract business system.

- **Low**

Low impact vulnerabilities are those that marginally affect smart contracts. These vulnerabilities may pose certain security risks to the contract business system and necessitate improvements, but they generally lead to lesser concerns compared to higher-level vulnerabilities.

3.1.4 Likelihood of Exploitation

- **Probable**



A probable likelihood indicates that exploiting the vulnerability requires minimal effort or resources, without any specific conditions needing to be met. The vulnerability can be exploited reliably and consistently.

- **Possible**

A possible likelihood suggests that exploiting the vulnerability involves some level of cost or meets certain prerequisites. The vulnerability may not be triggered easily or consistently.

- **Unlikely**

An unlikely likelihood implies that exploiting the vulnerability demands considerable resources or meets stringent conditions, making it notably challenging to exploit.

- **Rare**

A rare likelihood signifies that triggering the vulnerability requires an exceptionally high level of resources or meets conditions that are extremely difficult to fulfill, making exploitation highly improbable.

3.1.5 Fix Results Status

Status	Description
	The project party fully fixes a vulnerability.
Partially Fixed	The project party did not fully fix the issue, but only mitigated the issue.
Acknowledged	The project party confirms and chooses to ignore the issue.



3.2 Audit Categories

No.	Categories	Subitems
1	Coding Conventions	Compiler Version Security
		Deprecated Items
		Redundant Code
		require/assert Usage
		Gas Consumption
2	General Vulnerability	Integer Overflow/Underflow
		Reentrancy
		Pseudo-random Number Generator (PRNG)
		Transaction-Ordering Dependence
		DoS (Denial of Service)
		Function Call Permissions
		call/delegatecall Security
		Returned Value Security
		tx.origin Usage
		Replay Attack
		Overriding Variables
		Third-party Protocol Interface Consistency
3	Business Security	Business Logics
		Business Implementations
		Manipulable Token Price
		Centralized Asset Control
		Asset Tradability
		Arbitrage Attack



KEKKAI categorizes smart contract security issues into three distinct groups: Coding Conventions, General Vulnerability, and Business Security. Here's an overview of each category:

- **Coding Conventions**

This category assesses whether smart contracts adhere to established security coding guidelines specific to their programming language. For instance, contracts written in Solidity should lock in a specific compiler version and avoid using outdated or deprecated language features.

- **General Vulnerability**

General Vulnerability encompasses common security flaws that could be present in smart contract implementations. These vulnerabilities are typically intrinsic to the smart contract's nature and include issues like integer overflow/underflow or susceptibility to denial of service (DoS) attacks.

- **Business Security**

Business Security pertains to potential vulnerabilities directly linked to the unique business logic of each project, often exhibiting a higher degree of specificity. Examples include discrepancies between the code's lock-up terms and those outlined in the project's white paper or vulnerabilities like flash loan attacks stemming from incorrect oracle settings for price retrieval.

*It's important to note that projects may face risks associated with third-party protocols integrated into their system, which falls outside KEKKAI's purview. Business Security necessitates active involvement from the project team, who, along with their users, should remain vigilant at all times to mitigate risks effectively.



3.3 Disclaimer

The Audit Report produced by KEKKAI pertains specifically to the services outlined in the corresponding service agreement. It is intended for use solely by the Party receiving the service (hereafter referred to as the "Served Party") within the parameters and limitations set forth in the service agreement. It is not permissible for any third parties to transmit, divulge, cite, depend upon, or alter the Audit Report for any purpose.

KEKKAI's Audit Report is strictly focused on the code and should not be construed as an endorsement or validation of the project itself. It does not provide any assurances or guarantees regarding the complete integrity of the analyzed code, the team responsible for the code, the business model, or compliance with legal standards.

The insights provided in KEKKAI's Audit Report are based solely on the code as submitted by the Served Party and the technological capabilities accessible to KEKKAI at the time of the audit. However, due to inherent technological limitations and potential issues such as incomplete information, tampering, deletion, concealment, or subsequent modifications of the code by the Served Party, the report may not be able to fully identify all potential risks.

KEKKAI's Audit Report is not intended to offer investment guidance or recommendations for any project. It is designed as a thorough examination aimed at assisting our clients in enhancing the quality of their code and reducing the inherent risks associated with blockchain technology.



Code Is Law



3.4 About KEKKAI

KEKKAI provides a web3.0 anti-fraud security solution for the consumer side based in Japan. It now offers a product range that includes an anti-fraud browser extension and a mobile application, and solution for web3.0 security such as smart contract security audit and penetration test. The aim of KEKKAI is to build the security layer of Web3 for consumers. It provides not only a firewall for daily crypto trading but also an environment where users can browse the Web3 world with peace of mind. Since its launch in February 2023, KEKKAI has gained over 40,000 users all over the world using KEKKAI's product. It is now protecting more than \$200M of user asset from not being attacked, and many of projects for security auditing.



OxKEKKAI



<https://kekkai.io>

